

**-LAPORAN PEMROGRAMAN BERBASIS OBJECT
JOBSHEET 3**



Nama:Daffa Dwi Nugroho

NIM : 254107020140

Kelas: TI 2-A

D-IV TEKNIK INFORMATIKA

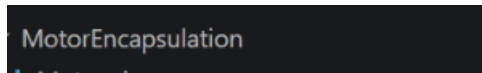
JURUSAN TEKNOLOGI INFORMASI

TAHUN AJARAN 2026

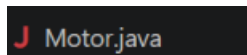
JOBSHEET 3

3.1 Percobaan 1 – Enkapsulasi

1. Buka Netbeans, buat project MotorEncapsulation.



2. Buat class Motor. Klik kanan pada package motorencapsulation – New – Java Class.



3. Ketikkan kode class Motor dibawah ini.

```
package MotorEncapsulation;

public class Motor {
    public int kecepatan = 0;
    public boolean kontakOn = false;

    public void printStatus(){
        if(kontakOn== true){
            System.out.println(x: "Kontak ON");
        } else {
            System.out.println(x: "Kontak OF");
        }
        System.out.println("kecepatan " + kecepatan + "\n");
    }
}
```

4. Kemudian buat class MotorDemo, ketikkan kode berikut ini

```
public class MotorDemo {
    Run | Debug
    public static void main(String[] args) {
        Motor motor = new Motor();
        motor.printStatus();
        motor.kecepatan = 50;
        motor.printStatus();
    }
}
```

5. Hasilnya adalah sebagai berikut:

```
ge\dfcada037b5396fee77f8ca2bfa
Kontak OF
kecepatan 0

Kontak OF
kecepatan 50
```

3.2 Percobaan 2 - Access Modifier

1. Pada class TestMobil, saat kita menambah kecepatan untuk pertama kalinya, mengapa muncul peringatan “Kecepatan tidak bisa bertambah karena Mesin Off!”?

- karena set default nya itu masi off jadi Ketika di tambahkan kecepatan tanpa menghidupkan mesin program tidak akan melakukan karena mesin masi mati

2. Mengapa atribut kecepatan dan kontakOn diset private?

- Atribut diset private untuk menerapkan prinsip enkapsulasi agar datanya tidak dapat diubah dan hanya bisa dimodifikasi melalui method yang telah disediakan.

3. Ubah class Motor sehingga kecepatan maksimalnya adalah 100!

```
}
public void tambahKecepatan(){
    if (kontakOn ==true) {
        if (kecepatan + 5 <= 100) {
            kecepatan += 5;
        } else {
            System.out.println(x: "Kecepatan sudah mencapai
            kecepatan = 100;
        }
    }
}
```

3.4 Percobaan 3 - Getter dan Setter

1. Class Anggota

```

package KoperasiGetterSetter;

public class Anggota {
    private String nama;
    private String alamat;
    private float simpanan;

    public void setNama(String nama){
        this.nama = nama;
    }
    public void setAlamat(String alamat){
        this.alamat = alamat;
    }
    public String getNama(){
        return nama;
    }
    public String getAlamat(){
        return alamat;
    }
    public float getSimpanan(){
        return simpanan;
    }
    public void setor(float uang){
        simpanan += uang;
    }
    public void pinjam(float uang){
        simpanan -= uang;
    }
}

```

Class KoperasiDemo

```

package KoperasiGetterSetter;

public class KoperasiDemo {
    Run | Debug
    public static void main(String[] args) {
        Anggota anggota1 = new Anggota();
        anggota1.setNama(nama: "Iwan Setiawan");
        anggota1.setAlamat(alamat: "Jalan Sukarno Hatta no 10");
        anggota1.setor(uang: 100000);
        System.out.println("Simpanan" + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());

        anggota1.pinjam(uang: 5000);
        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
    }
}

```

Hasil

```

SimpananIwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0

```

3.5 Percobaan 4 - Konstruktor, Instansiasi

1. Langkah pertama percobaan 4 adalah ubah class KoperasiDemo seperti berikut

```
Anggota anggota1 = new Anggota();  
System.out.println("Simpanan" + anggota1.getNama() + " : Rp"  
    anggota1.setNama(nama: "Iwan Setiawan");
```

2. Hasil dari program tersebut adalah sebagai berikut

```
PS C:\Users\Daffa Dwi\OneDrive\Documents>  
a KoperasiGetterSetter.KoperasiDemo  
Simpanannull : Rp0.0  
SimpananIwan Setiawan : Rp 100000.0  
Simpanan Iwan Setiawan : Rp 95000.0  
PS C:\Users\Daffa Dwi\OneDrive\Documents>
```

3. Ubah class Anggota menjadi seperti berikut

```
private float simpanan;  
Anggota(String nama, String alamat){  
    this.nama = nama;  
    this.alamat = alamat;  
    this.simpanan = 0;  
}
```

4. Selanjutnya ubah class KoperasiDemo sebagai berikut

```
Run | Debug  
: void main(String[] args) {  
    anggota1 = new Anggota(nama: "Iwan ", alamat: "Jalan Mawar")
```

5. Hasil dari program tersebut adalah sebagai berikut

```
SimpananIwan : Rp0.0  
SimpananIwan Setiawan : Rp 100000.0  
Simpanan Iwan Setiawan : Rp 95000.0  
PS C:\Users\Daffa Dwi\OneDrive\Documents>
```

3.6 Pertanyaan – Percobaan 3 dan 4

1. Apa yang dimaksud getter dan setter?

Getter adalah method untuk membaca nilai atribut private, sedangkan setter adalah method untuk mengubah nilai atribut tersebut.

2. Apa kegunaan dari method getSimpanan()?

Method getSimpanan() pada Anggota.java digunakan untuk mengembalikan dan membaca nilai dari atribut private simpanan

3. Method apa yang digunakan untuk menambah saldo?

Method yang digunakan untuk menambah saldo pada Anggota.java adalah setor(float uang). Method ini menjumlahkan saldo saat ini dengan nilai pada parameter uang.

4. Apa yang dimaksud konstruktor?

Konstruktor adalah method khusus yang otomatis dijalankan saat objek baru diciptakan untuk memberikan nilai awal pada atribut

5. Sebutkan aturan dalam membuat konstruktor?

Nama konstruktor harus sama persis dengan nama class tempatnya berada. Selain itu, konstruktor tidak boleh memiliki tipe data kembalian (return type), termasuk tidak boleh menggunakan keyword void.

6. Apakah boleh konstruktor bertipe private?

Boleh, konstruktor private digunakan untuk mencegah instansiasi objek dari luar class menggunakan keyword new.

7. Kapan menggunakan konstruktor dengan passing parameter?

Konstruktor ini digunakan ketika kita ingin langsung mengatur nilai awal yang spesifik untuk atribut saat objek diciptakan

8. Apa perbedaan inisialisasi atribut dan instansiasi atribut?

Instansiasi adalah proses penciptaan objek baru di memori menggunakan keyword new, seperti pembentukan objek anggota pada KoperasiDemo.java. Sebaliknya, inisialisasi adalah proses pemberian nilai pertama kali pada suatu atribut

9. Apa perbedaan inisialisasi method dan instansiasi method?

Method itu bukan sesuatu yang diciptakan (diinstansiasi) menjadi wujud baru, melainkan sekadar perintah atau aksi yang sudah ditulis (diinisialisasi) di dalam kode untuk dipanggil/dijalankan oleh objeknya.

5. Tugas

1. Cobalah program dibawah ini dan tuliskan hasil outputnya

```
EncapDemo.java > EncapDemo > setAge(int)
public class EncapDemo {
    private String name;
    private int age;

    public String getName(){
        return name;
    }
    public void setName(String newName){
        name = newName;
    }
    public int getAge(){
        return age;
    }
    public void setAge(int newAge){
        if (newAge > 30) {
            age = 30;
        }
        else {
            age = newAge;
        }
    }
}
```

```
EncapTest.java > EncapTest > main(String[])
1 public class EncapTest {
    Run | Debug
2     public static void main(String[] args) {
    click to add a breakpoint demo encap = new EncapDemo();
4         encap.setName(newName: "James");
5         encap.setAge(newAge: 35);
6
7         System.out.println("Name :" + encap.getName());
8         System.out.println("Age : " + encap.getAge());
9     }
10 }
```

```
Running Code (JSE) (Win
faf65d3\redhat.java\
Name :James
Age : 30
```

2. Pada program diatas, pada class EncapTest kita mengeset age dengan nilai 35, namun pada saat ditampilkan ke layar nilainya 30, jelaskan mengapa.

karena di awal itu udah ditetapkan atau di set ketika di inputkan angka diatas 30 itu angka umur nya itu langsung ke 30 kecuali jika diisikan dibawah 30 dia akan menghasilkan angka yang diinputkan

3. Ubah program diatas agar atribut age dapat diberi nilai maksimal 30 dan minimal 18

```
J EncapDemo.java
1  public class EncapDemo {
2      private String name;
3      private int age;
4
5      public String getName(){
6          return name;
7      }
8      public void setName(String newName){
9          name = newName;
10     }
11     public int getAge(){
12         return age;
13     }
14     public void setAge(int newAge){
15         if (newAge > 30) {
16             age = 30;
17         } else if (newAge < 18) {
18             age = 18;
19         }
20         else {
21
22         }
23     }
24 }
25
26
```

4. Pada sebuah sistem manajemen pergudangan kargo ekspedisi, terdapat class Kontainer yang memiliki atribut antara lain nomorResi, namaPemilik, kapasitasMaksimal (dalam kg), dan beratMuatanSaatIni. Kontainer dapat menerima tambahan muatan barang dengan batasan kapasitas maksimal yang telah ditentukan. Kontainer juga dapat diturunkan muatannya (bongkar muat). Ketika barang diturunkan, maka jumlah muatan saat ini akan berkurang sesuai dengan nominal berat yang dikeluarkan. Buatlah class Kontainer tersebut, berikan atribut (private), method getter, dan konstruktor sesuai dengan kebutuhan arsitektur enkapsulasi. Uji dengan kelas driver TestLogistik berikut ini untuk memeriksa apakah manajemen state kelas Anda telah berjalan dengan benar:

```

Kontainer.java > Kontainer > getNamaPemilik()
1  public class Kontainer {
2      private String nomorResi;
3      private String namaPemilik;
4      private double kapasitasMaksimal;
5      private double beratMuatanSaatIni;
6
7      public Kontainer(String nomorResi, String namaPemilik, double kapasitasMaksimal){
8          this.nomorResi = nomorResi;
9          this.namaPemilik = namaPemilik;
10         this.kapasitasMaksimal = kapasitasMaksimal;
11         this.beratMuatanSaatIni = 0.0;
12     }
13     public String getNamaPemilik(){
14     {
15         return namaPemilik;
16     }
17     public double getKapasitasMaksimal(){
18         return kapasitasMaksimal;
19     }
20     public double getBeratMuatanSaatIni(){
21         return beratMuatanSaatIni;
22     }
23     public void tambahMuatan(double berat) {
24         if (this.beratMuatanSaatIni + berat > this.kapasitasMaksimal) {
25             System.out.println("Maaf, berat muatan melebihi kapasitas maksimal kontainer.");
26         } else {
27             this.beratMuatanSaatIni += berat;
28         }
29     }

```

```

    }
    public void turunkanMuatan(double berat) {
        if (this.beratMuatanSaatIni - berat < 0) {
            this.beratMuatanSaatIni = 0.0;
        } else {
            this.beratMuatanSaatIni -= berat;
        }
    }
}

```

5. Modifikasi soal kargo logistik di atas agar nominal berat muatan yang dibongkar/diturunkan dalam satu kali pemanggilan method turunkanMuatan() maksimal hanya boleh sebesar 50% dari total berat muatan saat ini. Langkah ini diterapkan demi alasan keselamatan kerja operasional alat berat (crane). Jika operator mencoba menurunkan muatan melebihi batas 50% tersebut, sistem harus memblokir aksi dan memunculkan peringatan: "Maaf, demi keselamatan, pembongkaran muatan satu kali jalan tidak boleh melebihi 50% dari muatan saat ini!".

```

    }
    public void turunkanMuatan(double berat) {

        if (berat > (this.beratMuatanSaatIni * 0.5)) {
            System.out.println(x: "Maaf, demi keselamatan, pembongkaran muatan satu kali jalan tidak boleh melebihi 50% dari muatan saat ini!")
        } else {
            if (this.beratMuatanSaatIni - berat < 0) {
                this.beratMuatanSaatIni = 0.0;
            } else {
                this.beratMuatanSaatIni -= berat;
            }
        }
    }
}
}

```

6. Modifikasi kelas Main TestLogistik agar parameter jumlah berat barang yang dimasukkan (tambahMuatan) maupun berat barang yang dibongkar (turunkanMuatan) dapat menerima input nilai dinamis dari pengguna secara interaktif melalui terminal menggunakan utilitas java.util.Scanner

```
testLogistik.java > ...
import java.util.Scanner;
public class TestLogistik {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Kontainer kontainer = new Kontainer(nomorResi: "KNT-001", namaPemilik: "PT. Logistik Jaya", kapasitasMaksimal: 1000.0);
        boolean berjalan = true;
        System.out.println(x: "=== Sistem Manajemen Kargo Crane ===");
        System.out.println("Kapasitas Maksimal: " + kontainer.getKapasitasMaksimal() + " kg");
        while (berjalan) {
            System.out.println("\nMuatan Saat Ini: " + kontainer.getBeratMuatanSaatIni() + " kg");
            System.out.println(x: "Pilih Aksi:");
            System.out.println(x: "1. Tambah Muatan");
            System.out.println(x: "2. Turunkan Muatan");
            System.out.println(x: "3. Keluar");
            System.out.print(s: "Masukkan pilihan (1/2/3): ");
            int pilihan = input.nextInt();
            switch (pilihan) {
                case 1:
                    System.out.print(s: "Masukkan berat muatan yang ingin ditambahkan (kg): ");
                    double beratTambah = input.nextDouble();
                    kontainer.tambahMuatan(beratTambah);
                    break;
                case 2:
                    System.out.print(s: "Masukkan berat muatan yang ingin diturunkan (kg): ");
                    double beratTurun = input.nextDouble();
                    kontainer.turunkanMuatan(beratTurun);
                    break;
                case 3:
                    System.out.println(x: "Menuutup sistem operasional crane. Terima kasih.");
                    berjalan = false;
                    break;
                default:
                    System.out.println(x: "Pilihan tidak valid. Silakan coba lagi.");
            }
        }
        input.close();
    }
}
```

7. Sebuah aplikasi pemesanan tiket bioskop memerlukan kelas Tiket untuk mengelola data pemesanan secara aman. Kelas ini harus memiliki atribut private: judulFilm (String), hargaDasar (double), dan statusPembayaran (boolean). Ketentuan pengesetan nilai objek: • Konstruktor harus menerima parameter judulFilm dan hargaDasar. Nilai awal statusPembayaran selalu diset false (Belum Dibayar). • Atribut hargaDasar tidak boleh bernilai negatif. Jika input yang dimasukkan kurang dari 0, otomatis set nilai default ke Rp 35.000. •

Sediakan method lakukanPembayaran() untuk mengubah statusPembayaran menjadi true. • Nilai statusPembayaran hanya boleh dibaca (Read-Only) menggunakan getter, tidak boleh memiliki fungsi setter langsung dari luar kelas demi alasan keamanan transaksi. • Uji kode Anda menggunakan kelas TestBioskop berikut:

```
public class Tiket {
    private String judulFilm;
    private double hargaDasar;
    private boolean statusPembayaran;
    public Tiket(String judulFilm, double hargaDasar) {
        this.judulFilm = judulFilm;
        if (hargaDasar < 0) {
            this.hargaDasar = 35000.0;
        } else {
            this.hargaDasar = hargaDasar;
        }
        this.statusPembayaran = false;
    }
    public void lakukanPembayaran() {
        this.statusPembayaran = true;
    }
    public String getJudulFilm() {
        return judulFilm;
    }
    public double getHargaDasar() {
        return hargaDasar;
    }
    public boolean isStatusPembayaran() {
        return statusPembayaran;
    }
}
```

```
public class TestBioskop {
    Run | Debug
    public static void main(String[] args) {
        Tiket tiket1 = new Tiket(judulFilm: "Avengers: Endgame", -50000);

        System.out.println("Film: " + tiket1.getJudulFilm());
        System.out.println("Harga Tiket: " + tiket1.getHargaDasar());
        System.out.println("Status Lunas? " + tiket1.isStatusPembayaran());

        System.out.println(x: "\nMemproses pembayaran...");
        tiket1.lakukanPembayaran();
        System.out.println("Status Lunas Terbaru? " + tiket1.isStatusPembayaran());
    }
}
```

```
redhat.java\jdt_ws\JB_3_97104b4d\bin' 'Tes  
Film: Avengers: Endgame  
Harga Tiket: 35000.0  
Status Lunas? false
```

```
Memproses pembayaran...  
Status Lunas Terbaru? true
```